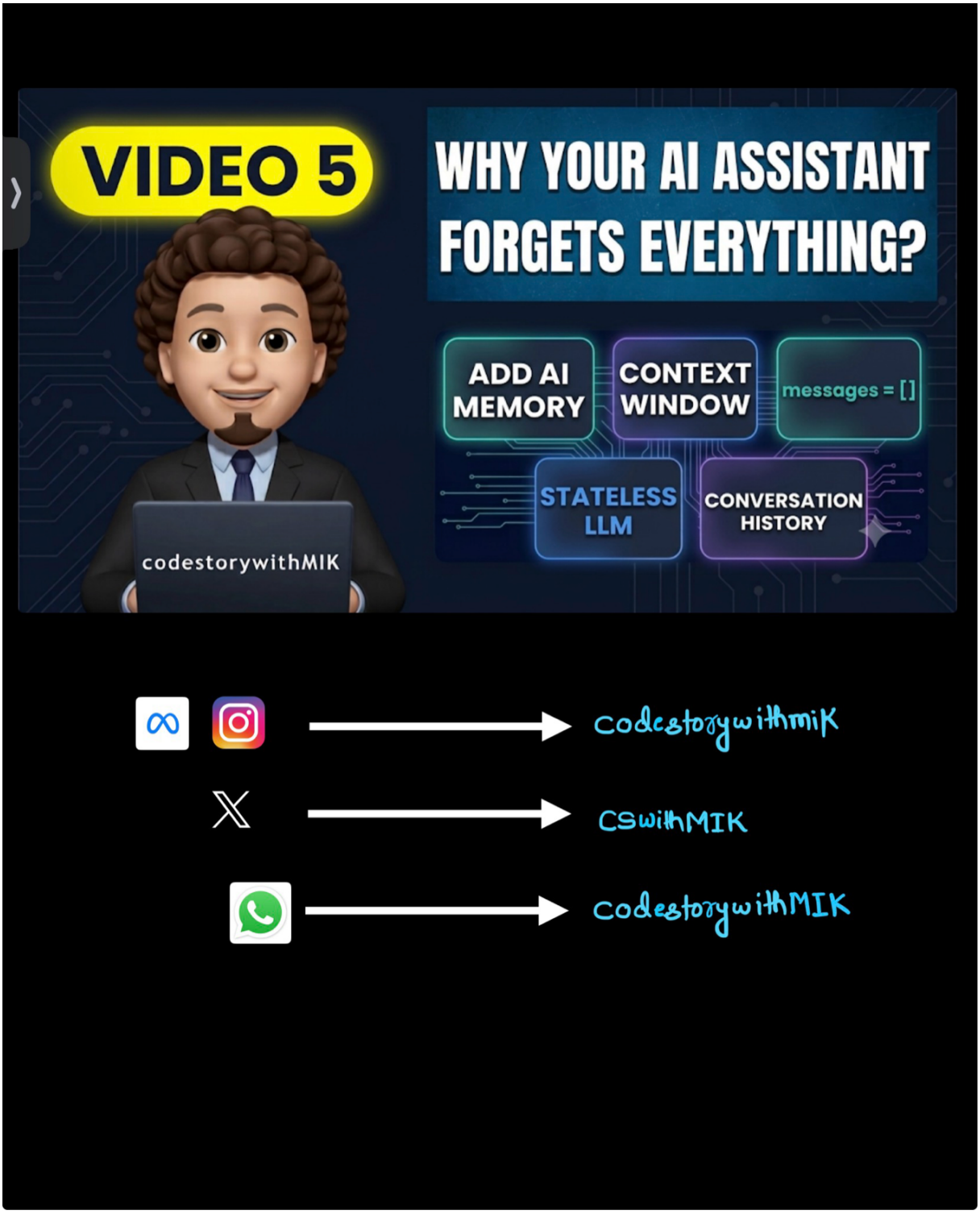
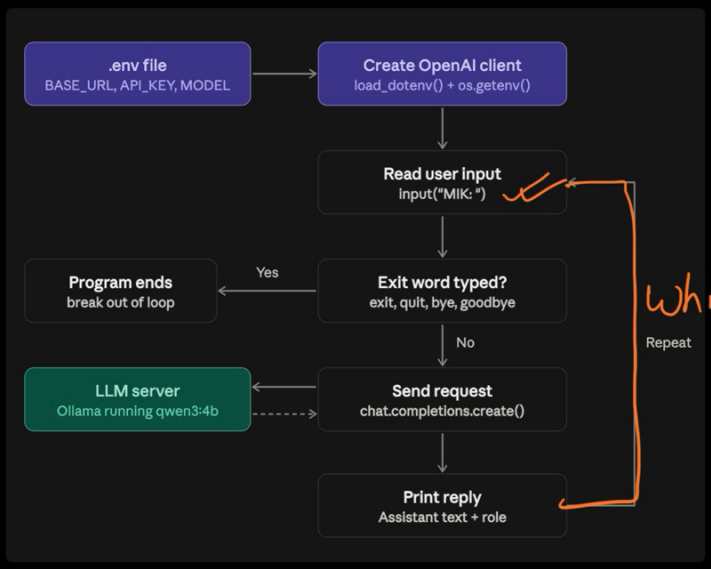

VIDEO 5
WHY YOUR AI ASSISTANT FORGETS EVERYTHING?
codestorywithMIK
ADD AI MEMORY
CONTEXT WINDOW
messages = []
STATELESS LLM
CONVERSATION HISTORY
codestorywithmik
CSwithMIK
codestorywithMIK

Pen

Video-4

Conversation Loop



Code

simple_ai_assistant.py > ...

```python
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv() #by default looks for .env file

client = OpenAI(
    base_url=os.getenv("BASE_URL"),
    api_key=os.getenv("API_KEY")
)

while True:

    user_input = input("\nMIK: ")

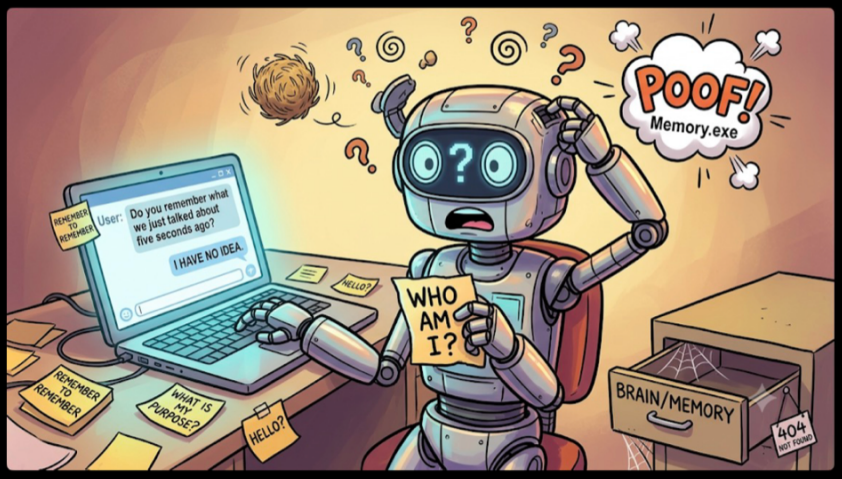
    if user_input.lower() in ["exit", "quit", "bye", "goodbye"]:
        break

    response = client.chat.completions.create(
        model=os.getenv("MODEL"),
        messages=[
            {
                "role":"user",
                "content": user_input
            }
        ]
    )
```

```
print("Asisstant: " + response.choices[0].message.content) #Response from the LLM model : qwen3:4b
print(response.choices[0].message.role)
```

↳ "assistant"

Our AI agent has no emory



Let's see a demo

But why it forgets?

"LLMs are stateless" ⇒ Remember for interview

- LLMs have no memory
- Every API call is brand new for it.

So, [LLM did not remember our previous call.
when the call finished, it's over for this call.]

→ STATELESS...

```
messages=[
    {
        "role": "user",
        "content": user_input
    }
```

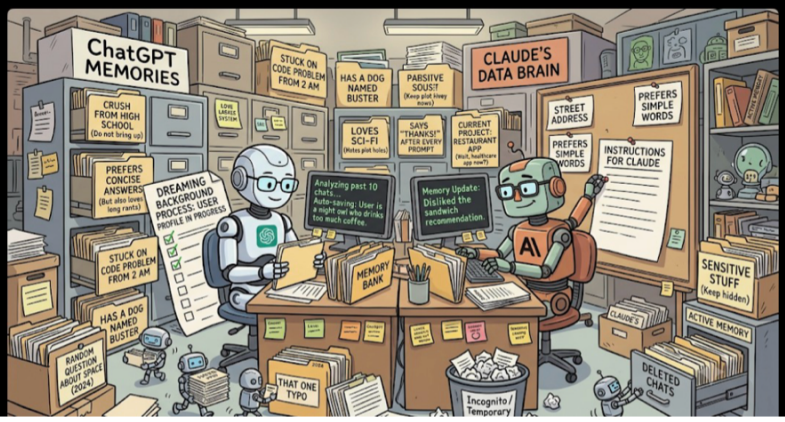
→ Everytime we send a

LLM treats

```
    {
        "role": "user",
        "content": user_input
    }
]
```

Everytime we send a message, LLM treats it as a new latest message.

Then H ChatGPT, claude, Gemini etc. remembers conversation ??????

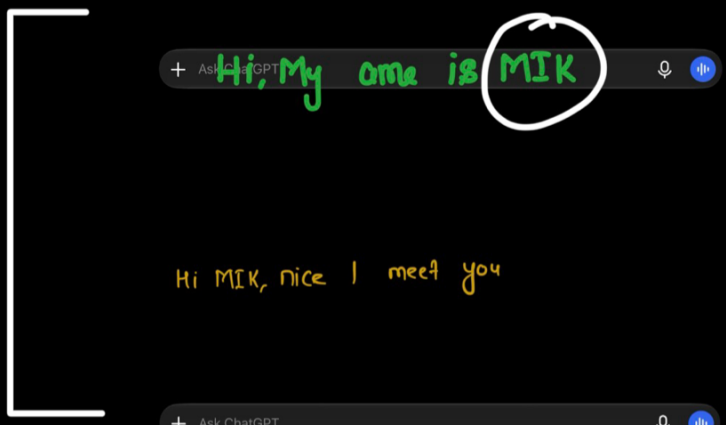


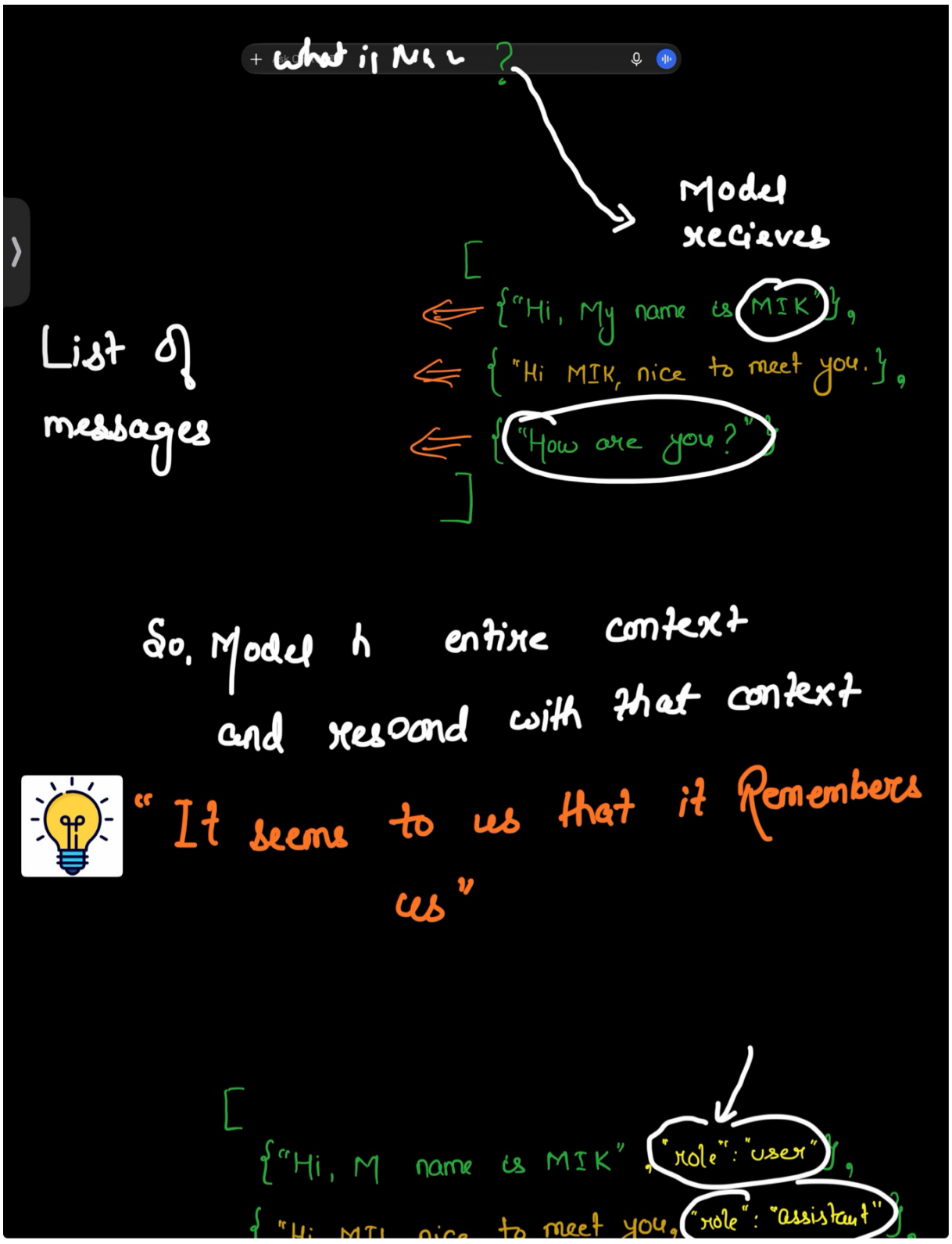
- Truth is - Model doesn't remember anything.



Claude/ hatGPT etc. they send the fixed conversation to the model every single time.






what is MIK ?
Model recieves
List of messages
[
{"Hi, My name is MIK"},
{"Hi MIK, nice to meet you."},
{"How are you?"}
]
So, Model h entire context and respond with that context
" It seems to us that it Remembers us "
[
{"Hi, M name is MIK" , "role": "user"},
{"Hi MI nice to meet you, "role": "assistant"},

```
[
  {"Hi, M name is MIK", "role": "user"},
  {"Hi MII, nice to meet you, "role": "assistant"},
  {"How  e you?", "role" : "user"}
]
```

Helps model understand, who said what.

You type message 1 :

Sent to odel : [msg1]

Model replies : reply1

You type messag 2 :

Sent to odel : [msg1, reply1, mg2]

Model replies : reply 2

You type message :

Sent to m del : [msg1, reply1, mg2, reply2, msg3]

Model replies : reply 3

You type message :

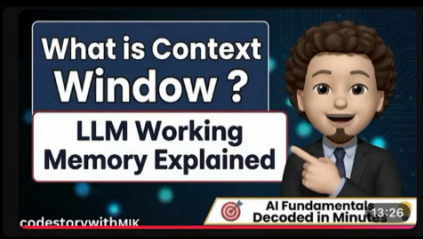
Sent to model : [msg1, reply1, msg2, reply2, msg3]

Model replies : reply3

[msg1, repl1, msg2, rep12, ms?, rep13]].

But There is a Catch :

[CONTEXT WINDOW]



Why Can't AI Remember Everything? The Truth About Context Windows | AI Fundamentals Decoded | MIK

codestorywithMIK

- AI models cannot read unlimited text at once.
- Every model has a maximum size of text it can

- Every model has a maximum size of text it can look at in one call.

  * limit = Context window
  * We used in tokens

↳ context length exceeded

↳ Silent cutting of older messages (loosing context)

↳ Quality of response drops.

↳ Slower response to large text size.

↳ If paid API → More tokens → More cost

ChatGPT, claude etc. uses several techniques to handle this problem.

1. Truncation / Sliding Window.

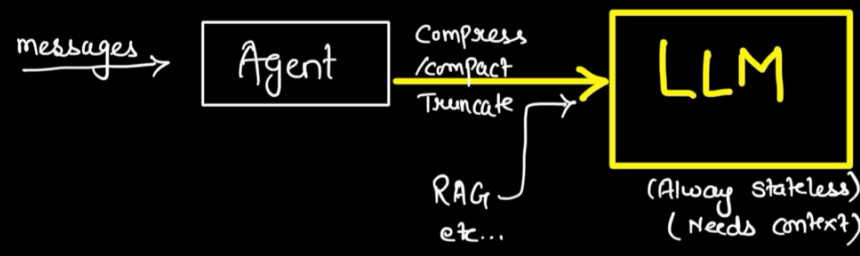


2. Summarization.

3. /compact (Present in claude code)
   only keeps key decisions, answers etc.

4. RAG

Notice : LLM is always Stateless



Apply same technique to add memory to our Agent...

simple_ai_assistant.py > ...

```python
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv() #by default looks for .env file

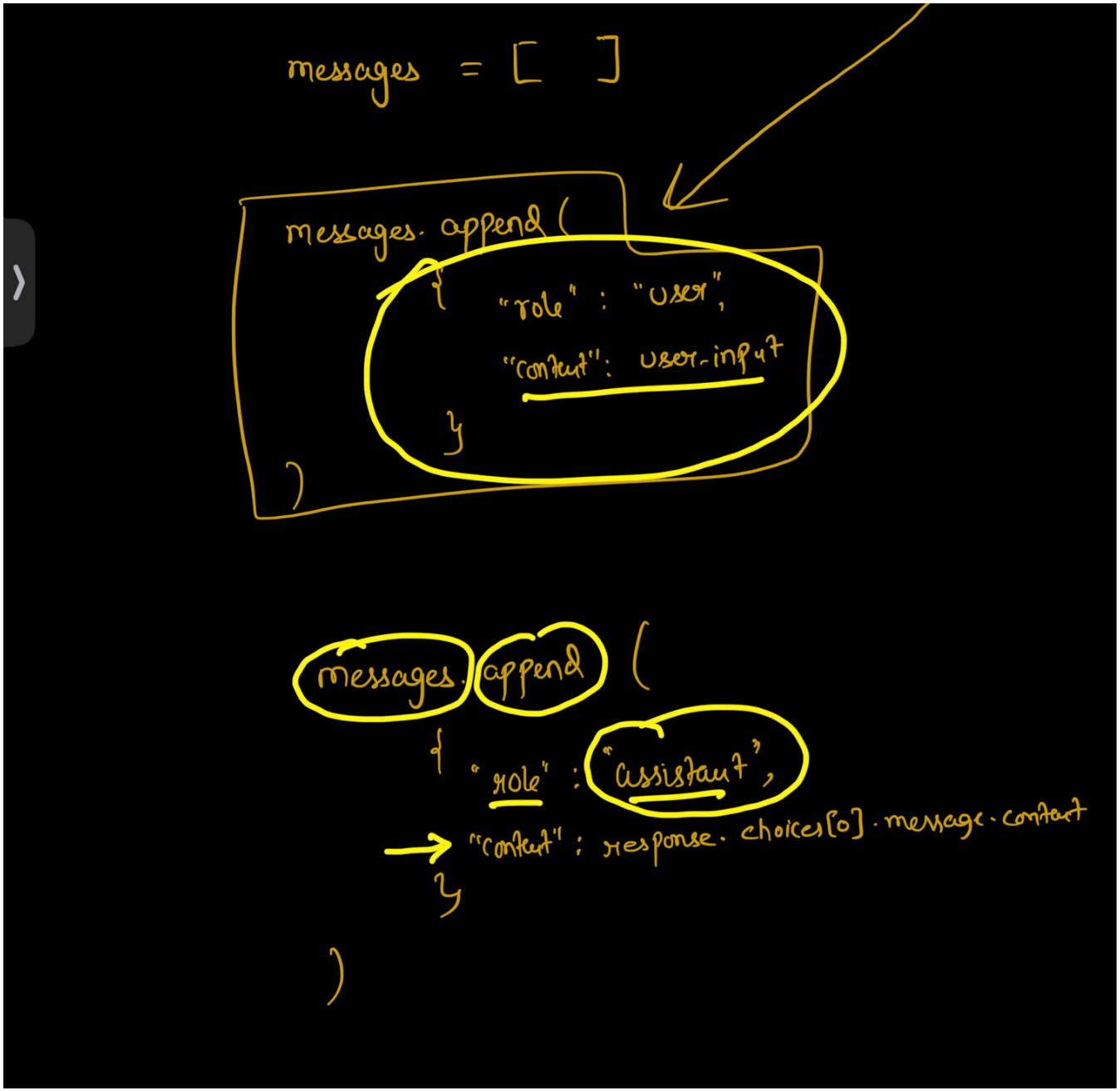
client = OpenAI(
    base_url=os.getenv("BASE_URL"),
    api_key=os.getenv("API_KEY")
)
messages = [ ]
while True:

    user_input = input("\nMIK: ")

    if user_input.lower() in ["exit", "quit", "bye", "goodbye"]:
        break

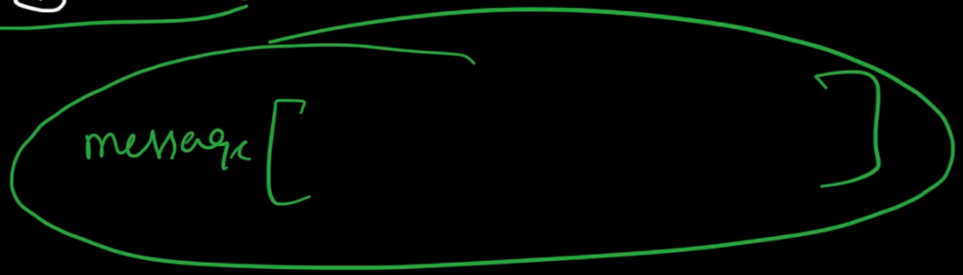
    response = client.chat.completions.create(
        model=os.getenv("MODEL"),
        messages=[ messages
            {
                "role":"user",
                "content": user_input
            }
        ]
    )

    print("Asisstant: " + response.choices[0].message.content) #Response from the LLM model : qwen3:4b
    print(response.choices[0].message.role)
```


messages = [ ]
messages.append(
{
"role" : "user",
"content": user_input
}
)
messages.append(
{
"role" : "assistant",
"content" : response.choices[0].message.content
}
)

Some Limitation...

1. Messages are lost when we close our Program. (Lives in RAM) → Later in Database/ file.
2. Keeps growing → context window hit
3. qwen3:4b (small models) → can miss details sometimes in a long history, even when they are present.



See you in the next video...